

SLIDE 1 — Meet Linear Search

What is Linear Search?

Linear Search is a method of finding a target by checking values one after another.

Imagine you're searching for your friend in a line:



You check:

First → **Second** → **Third** → **Fourth** → ...

That's the basic idea of Linear Search!

Example

Suppose we have:

10 25 40 15 30

Target = 15

Linear Search starts from the beginning:

10  → 25  → 40  → 15 

 Target found!

Easy Memory Trick

 **LINEAR = ONE BY ONE**

SLIDE 2 — How Does Linear Search Work?

Follow the Search Path

Linear Search moves through the values **in order**.

 Target = 40

[10] → [25] → [40] → [15] → [30]



What's Happening?

1 Start

Begin with the first value.



2 Compare

Ask:

 “Is this the target?”



3 Move

If it isn't the target, go to the next value.



4 Stop

If it matches →  **Found!**

Think Like the Computer

 Check



 Match?



YES NO



Found



Next
value

Remember

Check → Compare → Move → Repeat

SLIDE 3 — Where Can the Target Be?



The target doesn't always appear in the same position.

Let's see what happens in different situations.

Target at the Beginning

 10 25 40 15 30

 Found immediately!

Comparisons = 1

Target in the Middle

10 25  40 15 30

The computer checks some values first.

Comparisons = 3

Target at the End

10 25 40 15  30

Most values must be checked.

Comparisons = 5

✖ Target Not Present

10 25 40 15 30

 Target = 50

The computer checks everything:

10  → 25  → 40  → 15  → 30 
 Not Found

 Key Idea

 Where the target is can change how many values are checked.

● SLIDE 4 — Linear Search with Unsorted Data

🤔 Does the data have to be arranged?

No!

One useful feature of Linear Search is that the values **don't need to be sorted**.

 Example

35 8 72 14 50

 Target = 14

The values are not arranged from smallest to largest.

But Linear Search can still search:

35 ✖
↓
8 ✖

↓
72 ❌

↓
14 ✅

🎉 **Found!**

🌐 **Real-Life Example**

Imagine a classroom where students aren't standing according to roll number.

👤 23
👤 7
👤 31
👤 15
👤 4

If you're looking for roll number **15**, you can still check students one by one.

🧠 **Remember**

🔪 **Unsorted data?**

➡ Linear Search can still work! ✅

🟪 **SLIDE 5 — When Should We Use Linear Search?** ⚡

👍 **Linear Search is useful when...**

📦 **The data is small**

It's easy to check a small number of values.

🔪 **The data is unsorted**

You don't need to arrange the values first.

You need a simple search

The idea is easy:

Start → Check → Move → Find

But There Is a Catch...

Imagine searching through:

-  5 values →  Easy
-  50 values →  More checking
-  500 values →  More work

If the target is near the end—or isn't present—the computer may have to check **many values**.

BIG MEMORY TRICK

 **LINEAR SEARCH = CHECK ONE BY ONE**

LEVEL CLEARED!

You now understand:

-  What Linear Search means
-  How it checks values
-  How target position affects searching
-  Why it can work with unsorted data
-  When it is useful

QUESTIONS:

Got you bro 🔥 For **Ch2: Linear Search**, here are exactly **5 questions** in your requested pattern:

- **2** Fill-in-the-blank code
 - **1** MCQ with code
 - **1** Basic MCQ
 - **1** Code — fill one line
 - Every question has 💡 **Hint** + 📖 **Glossary**
 - Glossary only uses terms actually relevant to that question.
-

Ch2 — Linear Search Quiz

Q1. 💻 Fill in the Blank — Code

Complete the missing part to search for 25.

```
int numbers[] = {10, 25, 40, 15};  
  
int target = 25;
```

```
for(int i = 0; i < 4; i++) {  
    if(numbers[i] == _____) {  
        printf("Found");  
        break;  
    }  
}
```

💡 **Hint:** The blank should contain the value we are searching for.

📖 Glossary

- 🎯 **target** — The value we want to find.

✓ **Answer:** target

Q2. 🖥️ Fill in the Blank — Code

Complete the missing condition to search for 30.

```
int numbers[] = {10, 20, 30, 40};
```

```
int target = 30;
```

```
for(int i = 0; i < 4; i++) {  
    if(_____) {  
        printf("Found");  
        break;  
    }  
}
```

💡 **Hint:** Compare the current value with the target.

📖 Glossary

- 🎯 **target** — The value being searched for.

✓ **Answer:**

```
numbers[i] == target
```

Q3. 🧠 MCQ — Code

What will be printed by this code?

```
int numbers[] = {5, 10, 15, 20};
```

```
int target = 15;
```

```
for(int i = 0; i < 4; i++) {  
    if(numbers[i] == target) {  
        printf("Found");  
        break;  
    }  
}
```

A) Not Found

B) Found

C) 15

D) Nothing

 **Hint:** Check whether 15 exists in the values.

Glossary

-  **target** — The value the search is trying to find.

 **Answer: B) Found**

Q4. Basic MCQ

What is the main idea of **Linear Search**?

A) Divide the data into two parts

B) Check values one by one

C) Arrange values in order

D) Remove duplicate values

 **Hint:** Remember the Linear Search memory trick.

Glossary

-  **Linear Search** — A method that checks values one by one.

✓ **Answer: B) Check values one by one**

Q5. Code — Fill One Line

Complete the missing line so that the search stops when the target is found.

```
int numbers[] = {8, 12, 20, 25};
```

```
int target = 20;
```

```
for(int i = 0; i < 4; i++) {  
    if(numbers[i] == target) {  
        printf("Found");  
        _____;  
    }  
}
```

 **Hint:** Which statement immediately stops the loop?

Glossary

-  **loop** — A part of the program that repeats.
-  **target** — The value being searched for.

✓ **Answer:** break;